

Display Control Protocol

Version History

Version	Date	Author	Description
V1.0	2026-07-14	LIN	Initial release: frame format, playback control, color data management
V1.1	2026-08-25	LIN	Added device types (GEN); color type (16 colors, marker values changed from 0x21/0x37 to 0xFE/0xFF); added firmware upgrade (OTA) protocol (supported by some hardware): BIN image download, CRC verification, bootloader upgrade flow
V1.2	2026-08-31	LIN	Added 6.6 Set Slave Count (0x1A): payload, value range, effective timing, persistence, response behavior; corrected 6.1/6.3 playback interval descriptions (0x16: 1–255 s, 0 = default 15; 0x1D: default 1, 0 = 10), and noted the legacy 10-second-unit behavior of early HW_1 firmware
V1.3	2026-09-02	LIN	Added 6.7 Clear Power-On Auto-Play Configuration (0x05, temporarily retained): for clearing legacy data — auto-play parameters frozen in non-volatile storage by old firmware
V1.4	2026-09-23	LIN	Palette erratum: 0x05/0x06 names and reference colors swapped (0x05 = Green (0,200,0), 0x06 = Teal (64,224,208)); rewrote 7.4 per-segment pipeline (0x1F), delay values widened to uint16 (0–65535 frames); added 7.4.7 multi-device fleet timing constraints; 6.1/7.4.2 now note that the playback interval counts from the end of the refresh session

1. Overview

Devices communicate with the host program via **RS485 bus** or **USB CDC (virtual serial port)**.

1.1 Communication Interfaces

Interface	Parameters	Notes
RS485	115200 baud, 8 data bits, 1 stop bit, no parity	Bus topology, one master with multiple slaves
USB CDC	115200 baud (virtual serial port)	PC connected directly to the device's TYPE-C port

1.2 Addressing

Address	Description
0x00	PC (sender)

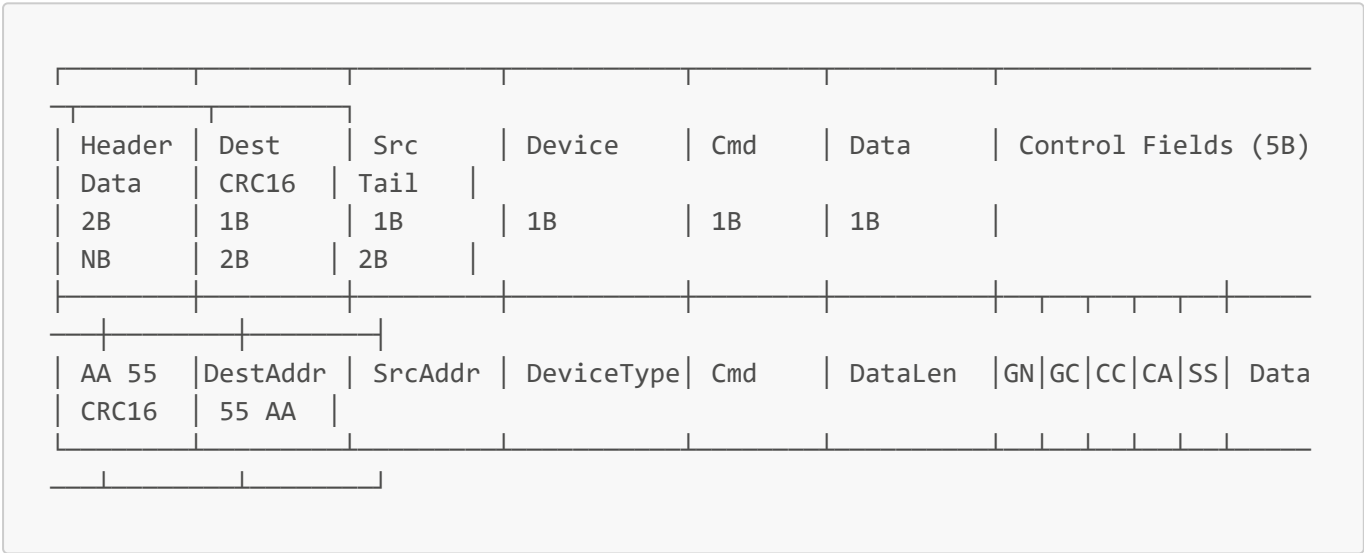
Address	Description
0x01	RS485 bus master (default)
0x02 ~ 0xFE	Slave addresses (set by DIP switch/configuration)
0xFF	Broadcast address

Note: The device at address 0x01 defaults to the RS485 bus master and is responsible for master/slave coordination. With broadcast commands (`dest_addr = 0xFF`), all devices act simultaneously but send no ACK.

2. Frame Format

2.1 Frame Structure

All communication uses a unified frame format, little-endian (multi-byte fields, low byte first).



2.2 Field Descriptions

Field	Length	Offset	Description
Frame header	2 bytes	0	Fixed value 0xAA 0x55 (big-endian: AA first, then 55)
Destination address (DestAddr)	1 byte	2	Receiving device address; 0xFF is broadcast
Source address (SrcAddr)	1 byte	3	Sender address; the PC is always 0x00
Device type (DeviceType)	1 byte	4	Display pattern type (see 3. Device Types)
Command (Cmd)	1 byte	5	Command code (see 4. Command Definitions)
Data length (DataLen)	1 byte	6	Number of data payload bytes (0–69; the maximum is the 69 bytes of the last 0x13 frame)

Field	Length	Offset	Description
Group number (GroupNo)	1 byte	7	Target group number (starting from 1)
Group count (GroupCount)	1 byte	8	Total number of system groups (1–255)
Segment count (ChipCount)	1 byte	9	Total number of data segments per slot (see 3. Device Types)
Segment index (ChipAddr)	1 byte	10	Index of the segment the current data belongs to (starting from 0)
Segment start (SegStart)	1 byte	11	Segment start number (usually 0)
Data payload (Data)	N bytes	12	Data carried by the command; length given by DataLen
CRC16	2 bytes	12+N	CRC16 checksum (low byte first)
Frame tail	2 bytes	14+N	Fixed value 0x55 0xAA (big-endian: 55 first, then AA)

2.3 CRC16 Verification

- **Algorithm:** Modbus CRC16
- **Polynomial:** **0x8005** (reversed **0xA001**)
- **Initial value:** **0xFFFF**
- **Calculation range:** From the destination address (inclusive) to the end of the data payload (inclusive), **10 + DataLen** bytes in total
- **Storage:** Low byte first (little-endian)

C reference implementation:

```
uint16_t CRC16(uint8_t *data, uint16_t len) {  
    uint16_t crc = 0xFFFF;  
    for (uint16_t i = 0; i < len; i++) {  
        crc ^= data[i];  
        for (uint8_t j = 0; j < 8; j++) {  
            if (crc & 0x0001)  
                crc = (crc >> 1) ^ 0xA001;  
            else  
                crc >>= 1;  
        }  
    }  
    return crc; // low byte first  
}
```

3. Device Types

The device type **DeviceType** determines the shape of the displayed pattern and the data layout.

Value	Name	Description
0x00	Brick wall pattern (WALL_BRICKS) (MCB01)	Wall bricks
0x01	Hexagon pattern (H_WALL_BRICKS) (MCB03)	Hexagon
0x03	Number board (NUMBER_BRAND) (MCB05)	Number board
0x04	Small honeycomb hexagon pattern (SMALL_H_WALL_BRICKS) (MCB02)	Small honeycomb hexagons
0x05	High-definition screen (HD_WALL_BRICKS) (MCB04)	High-resolution display
0x06	Generic (COMMON_KIND)	Generic device type, 62-byte solid color fill, commonly used for testing

Segmentation note: Color data (0x13)/pipeline (0x1F) is organized in "segments" of 64 bytes each. The number of segments per slot is determined by the device type (brick wall: 2 segments, others: 1; HD screen: 1–10). Segments are distinguished by **ChipAddr**.

4. Command Definitions

4.1 Color Data and Playback Control Commands

Command	Name	Description
0x13	Save color data	Save color data to the specified slot (supports multi-chip transfer in separate frames)
0x14	Delete specified slot	Delete the color data of the specified slot
0x15	Clear all colors	Clear the color data of all 20 slots
0x16	Start playback	Start playing from the specified slot
0x17	Stop playback	Stop current playback
0x1B	Set slot configuration	Set playback parameters of the specified slot (mode, direction, delays)
0x1D	Display single frame	Play the specified slot once, then stop without auto-advancing
0x1F	Send per-segment pipeline	Send the per-segment refresh start delay table of the specified slot (uint16 delay values, low/high half-frame per chip, independent of color data; value = offset in frames from refresh start, 0–65535; see 7.4 Send Per-Segment Pipeline Data (0x1F))

Command	Name	Description
0x25	Clear pipeline	Clear the pipeline data of the specified slot (color data is kept; see 7.5 Clear Per-Slot Pipeline (0x25))

4.2 System Management Commands

Command	Name	Description
0x01	Device reset	Reset the device
0x03	Set address	Change the device's RS485 address
0x04	Save configuration	Save the current configuration to storage
0x05	Clear power-on auto-play configuration	For clearing legacy data: clears the auto-play parameters frozen in storage by old firmware, restores defaults, and persists immediately (see 6.7 Clear Power-On Auto-Play Configuration (0x05))
0x1A	Set slave count	Master-only: set the number of RS485 slaves (see 6.6 Set Slave Count (0x1A))
0x26	Start OTA upgrade	Pass the image size and CRC to start an upgrade reception (see 11. Firmware Upgrade (OTA))
0x27	OTA image data	Download BIN image data chunks in frames (flags + offset + chunk)
0x28	End OTA upgrade	Notify that the image transfer is complete; after CRC verification passes, set the upgrade flag and reset
0x29	Query OTA status	Query the current OTA reception state (state/size/crc, resume reserved)

4.3 Response Commands

Command	Name	Description
0x80	ACK – Success	Command executed successfully
0x81	ACK – Failure	Command execution failed (data payload is a 1-byte error code)
0x82	ACK – Busy	Device busy, retry later
0x83	ACK – Invalid command	Unsupported command
0x84	ACK – CRC error	Frame CRC verification failed
0x85	ACK – Parameter error	Invalid command parameters

5. Color Data Format

5.1 Color Encoding

Colors are represented as single-byte index values.

The current palette defines only 0x00 ~ 0x0F; all other values are reserved. Do not use them.

5.1.1 Palette (0x00 ~ 0x0F, 16 Colors)

Value	Color	Reference (RGB) / Output behavior
0x00	White	(255, 255, 255)
0x01	Yellow	(255, 255, 0)
0x02	Blue	(0, 0, 255)
0x03	Red	(255, 0, 0)
0x04	Black	(0, 0, 0)
0x05	Green	(0, 200, 0)
0x06	Teal	(64, 224, 208)
0x07	Almond	(235, 210, 180)
0x08	Light pink	(255, 209, 220)
0x09	Sky blue	(135, 206, 235)
0x0A	Orange	(255, 140, 0)
0x0B	Yellow green	(154, 205, 50)
0x0C	Olive gray	(112, 116, 85)
0x0D	Brown	(139, 69, 19)
0x0E	Dark brown	(92, 51, 23)
0x0F	Smoke blue	(110, 130, 150)
0xFE	Start/end marker (not a color)	Fixed marker byte marking the start/end of the color data array (see each pattern's data format)
0xFF	Fill / no refresh (not a color)	That segment is not refreshed; used for empty positions

5.2 Brick Wall Pattern (0x00) Color Data Format

Each device has 2 chips, and each chip requires 64 bytes of color data. The data is sent in two transfers via **two arrays** (64 bytes each):

Array 1 (Chip 0):

```
[0xFE][62 bytes of color data][0xFF]
```

- Index 0: 0xFE (start marker)
- Indexes 1–62: color data

- Index 63: **0xFF** (padding)

Array 2 (Chip 1):

```
[0xFF][63 bytes of color data, ending with 0xFE]
```

- Index 0: **0xFF** (padding)
- Indexes 1–61: color data
- Index 62: **0xFE** (end marker)
- Index 63: data

5.3 Hexagon Pattern (0x01) Color Data Format

Single 64-byte array, 54 valid triangles, **number = array index** (each color is placed at the index position corresponding to the triangle number — a sparse, not contiguous, layout):

```
[0xFE][54 triangle colors (scattered by number across indexes 2–61)][missing positions filled with 0xFF][0xFE]
```

- Index 0: **0xFE** (start marker)
- Indexes 1–62: colors of the numbered triangles; non-existent numbers (1, 18–23, 62) are filled with **0xFF**
- Index 63: **0xFE** (end marker)

For the detailed pixel mapping, see [14. Appendix B: Hexagon Pattern \(0x01\) Pixel Mapping](#).

5.4 Small Honeycomb Hexagon Pattern (0x04) Color Data Format

Single 64-byte array, 60 valid hexagon color points (numbered 1–62, with 12 and 52 missing):

```
[0xFF][colors 1–11][horizontal separator color][colors 13–51][horizontal separator color][colors 53–62][0xFE]
```

- Index 0: **0xFF** (padding)
- Indexes 1–11: colors of numbers 1–11
- Index 12: horizontal separator color
- Indexes 13–51: colors of numbers 13–51
- Index 52: horizontal separator color
- Indexes 53–62: colors of numbers 53–62
- Index 63: **0xFE** (end marker)

5.5 Number Board (0x03) Color Data Format

Single 64-byte array, 60 numbered digit mappings, **digit number = array index**:

```
[0xFE][60 digit color data][2 bytes padding][0xFE]
```

- Index 0: 0xFE (start marker)
- Indexes 1–60: colors of digits 1–60
- Indexes 61–62: padding
- Index 63: 0xFE (end marker)

5.6 Generic (0x06) Color Data Format

Single 64-byte array, filled with color codes:

```
[0xFE][62 color data bytes][0xFE]
```

- Index 0: 0xFE (start marker)
- Indexes 1–62: 62 identical color codes (see 5.1 Color Encoding)
- Index 63: 0xFE (end marker)

6. Playback Control

6.1 Start Playback (0x16)

Start loop playback from the specified slot. Broadcast command; all devices start in sync.

Data payload (1–2 bytes):

Offset	Length	Description
0	1 B	Start slot number (0–19)
1	1 B	Playback interval (seconds, optional; valid range 1–255, 0 = default 15)

Send example (hex):

Header	Dst	Src	Type	Cmd	Len	GN	GC	CC	CA	SS	Data	CRC	Tail
AA 55	FF	00	00	16	02	01	01	02	00	00	00 05	FC 7C	55 AA

interval 5 s)

0)

||

|

└─ Data[1] = 5 (playback

└─ Data[0] = 0 (start slot

Playback interval note: The playback interval unit is **seconds**. The interval counts from the **end of the slot's refresh session** (not from the start of refreshing); when a slot carries a large-delay pipeline table, the actual round period $\approx$ max delay + interval (see 7.4.2 / 7.4.7). Compatibility note: early HW_1 firmware interprets this byte in **10-second units** (receiving 10 plays for 100 s). On a bus mixing HW_1

units, the sender must convert for that firmware or upgrade it to the seconds-based version.

Compatibility note: when using large-delay tables or long intervals (> 15 s), all devices on the bus must run V1.4 or later firmware **fleet-wide** (early firmware has abnormal round rhythm in this scenario); do not mix versions.

Device response:

Response	Description
ACK_SUCCESS (0x80)	Playback started
ACK_FAIL (0x81) + error code	Start failed (invalid slot, etc.)

6.2 Stop Playback (0x17)

Stop current playback immediately. Broadcast supported.

Data payload: Empty (**DataLen** = 0)

Send example (hex):

```
Header  Dst Src Type Cmd Len GN GC CC CA SS  CRC   Tail
AA 55   FF 00 00  17  00  01 01 02 00 00  XX XX  55 AA
```

Device response:

Response	Description
ACK_SUCCESS (0x80)	Playback stopped

6.3 Display Single Frame (0x1D)

Play the specified slot once, then stop without auto-advancing to the next slot. Used for "single-frame preview" scenarios.

Data payload (1–2 bytes):

Offset	Length	Description
0	1 B	Slot number (0–19)
1	1 B	Playback interval (seconds, optional; default 1, 0 = 10)

6.4 Playback Mode (DisplayMode)

Value	Name	Description
0x00	Pixel-by-pixel	Each pixel lights up in turn (deprecated)
0x01	Pair-by-pair	Two pixels light up as a group (deprecated)

Value	Name	Description
0x02	All at once	All pixels light up simultaneously (default, recommended)

6.5 Playback Direction (DisplayDirection)

Value	Name	Description
0x00	Top to bottom	
0x01	Bottom to top	
0x02	Left to right	
0x03	Right to left	
0x04	Outside in	
0x05	Inside out	
0x06	Clockwise	
0x07	Counterclockwise	
0x08	Clockwise + inverted delay	Clockwise with an inverted-color effect
0x09	Counterclockwise + inverted delay	Counterclockwise with an inverted-color effect
0x0A	Natural direction	Last-pixel direction (default, recommended)

Playback direction can be replaced by a custom PIPELINE array.

6.6 Set Slave Count (0x1A)

Set the number of slaves on the RS485 bus. **Master-only command**: only the bus master (address 0x01) processes it; when sent to a slave address, the slave replies with a parameter-error response and its own configuration is unchanged.

Data payload (1 byte):

Offset	Length	Description
0	1 B	Slave count (0–253)

Send example (hex):

```

Header  Dst Src Type Cmd Len GN GC CC CA SS Data CRC   Tail
AA 55   01 00 00  1A  01  01 01 02 00 00 04   XX XX 55 AA
                                     |
                                     └─ Data[0] = 4 (slave count
4)

```

Note:

- Send unicast to the master (**DestAddr** = 0x01); on broadcast, likewise only the master executes it, but no response will be received.
- The group/chip fields (GN/GC/CC/CA/SS) of this command are not validated and may contain any legal values.

Effective timing: Effective immediately; no reboot required. The master coordinates subsequent playback with the new count; a count of 0 means standalone mode (waiting for no slaves).

Persistence: Written to non-volatile storage on success; survives power loss and is loaded automatically after reboot.

Device response:

Response	Description
ACK_SUCCESS (0x80)	Set successfully (written to storage)
ACK_FAIL (0x81) + 0x0A (parameter error)	Target is not the master, or the count is out of 0–253
ACK_FAIL (0x81) + 0x06 (hardware error)	Storage write failed
ACK_FAIL (0x81) + 0x04 (data length error)	Data payload is empty

6.7 Clear Power-On Auto-Play Configuration (0x05)

Compatibility command (temporarily retained since V1.3), for clearing legacy data. This command specifically clears **auto-play parameters frozen in non-volatile storage by old firmware** (legacy data such as enabled / start slot / interval / mode snapshot-persisted along with 0x1A/0x04, e.g., interval 10 s), restores defaults in one shot, and persists them so the old values can never become stuck.

Data payload: Empty (0 bytes).

Send example (hex):

Header	Dst	Src	Type	Cmd	Len	GN	GC	CC	CA	SS	Data	CRC	Tail
AA 55	01	00	00	05	00	01	01	02	00	00	(none)	XX XX	55 AA

Note:

- Both master and slaves can process it (slaves' storage may also hold frozen legacy data); send unicast to the target device.
- The group/chip fields (GN/GC/CC/CA/SS) of this command are not validated and may contain any legal values.

Execution behavior: Restores the 4 auto-play fields to defaults — **enabled=0**, start slot 0, interval 15 s, loop mode — and **persists the full snapshot immediately** (no need to send 0x04 again).

Effective timing: Immediate. Power-on behavior after clearing: the master (address 0x01) still unconditionally auto-plays start slot 0 at interval 15 s on power-up (firmware default); slaves do not auto-play and follow the

master's broadcast. Daily playback intervals are carried on-site by the 0x16 start-playback command and do not depend on this configuration.

Device response:

Response	Description
ACK_SUCCESS (0x80)	Cleared successfully (defaults written to storage)
ACK_FAIL (0x81) + 0x06 (hardware error)	Storage write failed (RAM already restored to defaults; old values persist across power cycles)

Compatibility note: Old firmware (V1.2 and earlier) does not recognize 0x05 and replies ACK_FAIL (0x81) + 0x0A — the device must first be upgraded to V1.3 or later firmware before clearing. This command exists to clear legacy stock data and may be removed in a later version once the legacy fleet is cleaned up.

7. Color Data Management

7.1 Send Color Data (0x13)

Save color data into the storage of the specified slot. **All device types share one storage area; each slot is stored independently;** sending new data overwrites the old data.

7.1.1 Basic Concepts

Concept	Description
Slot	Where color data is stored; 20 in total (numbered 0–19)
Segment (Chip)	Each slot's data is organized in 64-byte units and may span multiple segments (the field names ChipAddr/ChipCount are legacy)
Group	Devices on the RS485 bus, one address per group
Frame transfer	With multiple segments, each segment's data is sent in its own frame

7.1.2 Data Payload Format

Intermediate frame (2 + 64 = 66 bytes):

Offset	Length	Description
0	1 B	Slot number (0–19)
1	1 B	Flags (see below)
2 ~ 65	64 B	Color data of one chip

Last frame (2 + 64 + 3 = 69 bytes):

Offset	Length	Description
0	1 B	Slot number
1	1 B	Flags (bit0 = 1 marks the last frame)
2 ~ 65	64 B	Color data of one chip
66	1 B	Playback mode (see 6.4 Playback Mode)
67	1 B	Playback direction (see 6.5 Playback Direction)
68	1 B	Pipeline delay in frames (default 1)

7.1.3 Flags

Bit	Constant	Description
bit0	0x01 (LAST_FRAME)	1 = last frame, triggers the storage write
bit1	0x02 (CLEAR_OLD)	1 = clear old data before writing (reserved)
bit2–7	—	Reserved

7.1.4 Send Procedure

Taking the brick wall pattern (2 segments) as an example:

```

Step 1: PC sends the intermediate frame of segment 0
  → Header: AA 55
  → Destination address: device address (e.g., 0x01)
  → Device type: 0x00
  → Command: 0x13
  → Data length: 66 (0x42)
  → Segment index (ChipAddr): 0x00
  → Data: [slot_no][flags=0x00][64 bytes of color data]
  → Device response: ACK_SUCCESS (0x80)

Step 2: PC sends the last frame of segment 1
  → Header: AA 55
  → Destination address: device address
  → Device type: 0x00
  → Command: 0x13
  → Data length: 69 (0x45)
  → Segment index (ChipAddr): 0x01
  → Data: [slot_no][flags=0x01][64 bytes of color data][display_mode]
  [display_direction][pipeline_delay]
  → Device internals: CRC → assemble all segment data → write to storage
  → Device response: ACK_SUCCESS (0x80)

```

Note: After each send, wait for the device to reply **ACK_SUCCESS** before sending the next frame.

Old data clearing: When new data is sent to a slot that already has data, the old data is overwritten directly.

7.1.5 Segment Addresses and Groups

- **ChipAddr:** indicates which segment the current frame's color data belongs to (starting from 0); maximum value is **ChipCount - 1**
- **GroupNo:** indicates which RS485 group the data is sent to (device address = group number)
- Internally, the device buffers the data by **ChipAddr**; when the last frame arrives, all segments are assembled and written to storage

7.2 Delete Color Data of a Specified Slot (0x14)

Delete the color data of the specified slot, freeing storage space.

Data payload (1 byte):

Offset	Length	Description
0	1 B	Slot number (0–19)

Send example (hex), deleting slot 3:

Header	Dst	Src	Type	Cmd	Len	GN	GC	CC	CA	SS	Data	CRC	Tail	
AA 55	01	00	00	14	01	01	01	02	00	00	03	XX XX	55 AA	

Device response:

Response	Description
ACK_SUCCESS (0x80)	Deleted successfully
ACK_FAIL (0x81)	Deletion failed (hardware error)

7.3 Clear All Color Data (0x15)

Clear the color data of all 20 slots.

Data payload: Empty (**DataLen = 0**)

Send example (hex):

Header	Dst	Src	Type	Cmd	Len	GN	GC	CC	CA	SS	CRC	Tail	
AA 55	01	00	00	15	00	01	01	02	00	00	XX XX	55 AA	

Device response:

Response	Description
ACK_SUCCESS (0x80)	Cleared successfully
ACK_FAIL (0x81)	Clearing failed (hardware error)

Note: This command only clears color data, not the slot playback parameter configuration.

7.4 Send Per-Segment Pipeline Data (0x1F)

Send a **per-segment refresh start delay table** for the specified slot: one delay value per segment, specifying **on which frame after the refresh starts each segment begins refreshing** when the slot is displayed, thereby controlling refresh order and rhythm (top-to-bottom row sweeps, center-outward spread, forward/reverse — any order can be expressed). Independent of the color data (0x13), stored paired per slot; at display time, if the slot has pipeline data it is refreshed with the custom timing, otherwise the default timing is used (see 7.4.2).

Difference from the scalar pipeline_delay: the pipeline_delay in 0x13/0x1B is a single unified delay step for the whole slot; 0x1F sends a **per-segment** delay table — the two axes are orthogonal. Without a table, the default delay = pipeline_delay × segment number (see 7.4.2); with a table, pipeline_delay does not participate in the computation.

7.4.1 Data Payload Format

Delay values are **uint16 (little-endian, 0–65535 frames)**, split into **low/high half-frames** sent as two frames, each 2 + 64 = 66 bytes (same envelope as 0x13; MAX_DATA_LEN unchanged):

Offset	Length	Description
0	1 B	Slot number (0–19)
1	1 B	Flags (bit0 = 1 last frame; bit1 = 1 this frame is the high half-frame; remaining bits reserved, fill 0)
2 ~ 65	64 B	The low bytes (bit1=0) or high bytes (bit1=1) of this chip's delay table: array index = segment number (one-to-one with the 0x13 color array indexes)

Assembly rule: segment N's delay value = (high half-frame[N] << 8) | low half-frame[N].

- With multiple segments, send each segment's (chip's) low half-frame first, then the high half-frame, distinguished by ChipAddr; the last frame (the table's closing frame, i.e., the last chip's high half-frame) sets bit0=1, or writing is triggered when all chips' low/high half-frames are collected.
- Old-format compatibility:** sending only the low half-frame (legacy uint8 single-frame style, bit1=0) is also accepted; the high byte is filled with 0, equivalent to the original uint8 semantics; new code should always send both frames.
- Unlike 0x13: 0x1F has **no** playback mode/direction/pipeline_delay tail fields (pipeline is unrelated to playback parameters); every frame is 66 bytes.

7.4.2 Delay Value Semantics

- **Absolute offset:** segment N starts refreshing on the `delay[N]`-th frame after the slot's refresh start time T0 (T0 $\approx$ the moment refreshing starts after the `0x1D/0x16` display command is received). Segments with equal values start on the same frame.
- **Unit and range:** 1 unit = 1 display frame (nominally 10 ms/frame), range 0–65535 frames $\approx$ 0–655 s. **Exact inter-segment timing is subject to measurement** (frame counts are the protocol truth; the ms conversion is nominal).
- **Indexes 0 and 63 are ignored by firmware** (forced to the common terminal, no refresh); just fill these two bytes with 0.
- **Color takes priority:** segments whose color is `0xFF` (no refresh) are never refreshed regardless of the delay value; their delay bytes can be filled with 0.
- **Default timing without a table:** `delay[N] = pipeline_delay × N` (`pipeline_delay` defaults to 1, i.e., sequential per-segment refresh); same after clearing with `0x25`.
- **Direction ignored when a table exists:** when the slot has pipeline data, the playback direction set via `0x1B` does not participate; the order is decided entirely by this table.
- **Whole-slot duration** $\approx$ max delay + single-segment refresh time (i.e., the refresh session length); each segment's refresh waveform is unaffected.
- **Round rhythm:** the playback interval (0x16) counts from the **end** of the refresh session, so the round period of a slot with a table $\approx$ **max delay + playback interval**; a slot without a table has a session of about 2–3 s and a period $\approx$ interval. For multi-device fleet synchronization rules, see 7.4.7.

7.4.3 Send Procedure (2 Segments Example)

```

Step 1: PC sends segment 0's low half-frame
→ Command: 0x1F, data length: 66, segment index (ChipAddr): 0x00
→ Data: [slot_no][flags=0x00][64-byte low-byte table]
→ Device response: ACK_SUCCESS (0x80)

Step 2: PC sends segment 0's high half-frame
→ Command: 0x1F, data length: 66, segment index (ChipAddr): 0x00
→ Data: [slot_no][flags=0x02][64-byte high-byte table]
→ Device response: ACK_SUCCESS (0x80)

Step 3: PC sends segment 1's low half-frame
→ Command: 0x1F, data length: 66, segment index (ChipAddr): 0x01
→ Data: [slot_no][flags=0x00][64-byte low-byte table]
→ Device response: ACK_SUCCESS (0x80)

Step 4: PC sends segment 1's high half-frame (last frame)
→ Command: 0x1F, data length: 66, segment index (ChipAddr): 0x01
→ Data: [slot_no][flags=0x03][64-byte high-byte table]
→ Device internals: all segments collected → write to storage
→ Device response: ACK_SUCCESS (0x80)

```

7.4.4 Data Example (Single-Chip Device, Two Frames)

Scenario: a DeviceType 0x06 (GEN KIND, ChipCount=1) device, slot 19. Valid segments 1–6 are sewn on garment rows 3, 1, 2, 2, 5, 4 (row 1 at the top); we want a **top-to-bottom sweep with 100 ms between rows**

(the ms column below is a nominal conversion at 10 ms/frame; exact timing should be calibrated by measurement):

Seg (array index)	Row	Expected start	Delay (frames)	Low byte	High byte
2	1	T0 + 0	0	0x00	0x00
3, 4	2	T0 + 100 ms	10	0x0A	0x00
1	3	T0 + 200 ms	20	0x14	0x00
6	4	T0 + 300 ms	30	0x1E	0x00
5	5	T0 + 400 ms	40	0x28	0x00
7 ~ 62	—	color 0xFF, no refresh	fill 0	0x00	0x00
0, 63	—	ignored by firmware	fill 0	0x00	0x00

Frame 1 — low half-frame (flags = 0x00, 64-byte low-byte table, indexes 0 → 63):

```

Hdr      Dst Src Type Cmd Len GN GC CC CA SS Slot Flags Low-byte table (64B)
CRC      Tail
AA 55 01 00 03 1F 42 01 01 01 00 00 13 00 00 14 00 0A 0A 28 1E 00 x 57
XX XX 55 AA

```

Frame 2 — high half-frame (flags = 0x03 = last frame + high half-frame, 64 bytes all 0):

```

Hdr      Dst Src Type Cmd Len GN GC CC CA SS Slot Flags High-byte table (64B)
CRC      Tail
AA 55 01 00 03 1F 42 01 01 01 00 00 13 03 00 00 00 00 00 00 00 00 x 57
XX XX 55 AA

```

(DataLen = 0x42 = 66; the CRC16 calculation range follows the general rule)

Effect: segment 2 starts at T0, segments 3/4 at T0+100 ms, segment 1 at T0+200 ms, segment 6 at T0+300 ms, segment 5 at T0+400 ms — the whole board changes row by row from top to bottom. When multiple devices each hold their own delay table, the whole fleet changes row by row in sync at the same moments.

Device response: **ACK_SUCCESS** (0x80) (each of the two frames is acknowledged).

7.4.5 Storage and Lifecycle

- **Storage:** an independent storage area per slot, managed internally by the device
- **Paired with color data:** recommended to send 0x13 first to store the color data, then 0x1F to store that slot's pipeline (same slot number)
- **Deletion coupling:** 0x14 (delete one slot) / 0x15 (clear all) also clears the corresponding pipeline data
- **Backward compatibility:** when a slot has no pipeline data, display automatically falls back to the default timing; legacy flows are unaffected

- **Firmware upgrade:** after upgrading to the uint16 version (V1.4 firmware), pipeline tables stored by older firmware in uint8 format are automatically invalidated (data version mismatch); those slots fall back to the default timing and must be re-sent in the new format

7.4.6 Display Activation Conditions

The pipeline's segment count (ChipCount) must match the segment count of the slot's color data; otherwise it is treated as mismatched and degraded to the default timing. When displaying a single frame (**0x1D**) or playing (**0x16**) that slot, it is loaded and applied automatically; no extra command is needed.

7.4.7 Multi-Device Fleet Timing Constraints

In a master/slave fleet, large pipeline delays lengthen both the "session" and the "round". The following are automatic device-side behaviors and sender guidelines:

- **Round composition:** slaves report completion **immediately after finishing their refresh, with no dwell**; the master starts dwelling after its own session ends (the **0x16** interval); once the dwell elapses and all slaves have reported completion (or the completion wait times out), it broadcasts the switch to the next slot. Next-slot time $\approx \text{max}(\text{master session} + \text{interval}, \text{slowest session in the fleet})$; with the whole fleet on the same table this is **max delay + interval**.
- **Completion-wait fallback:** after the master itself completes, it waits at most **20 s** for slave completion frames; on timeout, the slave is treated as offline and the slot is force-switched. This yields the **slow-table tolerance**: when a slave's max delay exceeds the master's, the excess must not exceed **playback interval + 20 s** (≈ 35 s at a 15 s interval), otherwise that slave is switched early.
- **Firmware version:** when using large delays/long intervals, all devices on the bus must run V1.4 or later firmware; do not mix versions. After a firmware upgrade, per-slot pipeline tables must be re-sent (see 7.4.5).

7.5 Clear Per-Slot Pipeline (0x25)

Clear the pipeline data of the specified slot, **keeping the color data untouched**. After clearing, the slot's display falls back to the MCB library default timing (**has_pipeline=0**).

Difference from 0x14/0x15: **0x14/0x15** clear the color data along with it (pipeline cleared by coupling); **0x25** clears **only the pipeline** — the color data is unaffected.

Data payload (1 byte):

Offset	Length	Description
0	1 B	Slot number (0–19)

Send example (broadcast, clearing slot 0's pipeline):

Header	Dst	Src	Type	Cmd	Len	GN	GC	CC	CA	SS	Data	CRC	Tail
AA 55	FF	00	00	25	01	01	01	01	00	00	00	XX XX	55 AA

Device response:

Response	Description
ACK_SUCCESS (0x80)	Cleared successfully
ACK_FAIL (0x81)	Clearing failed (hardware error)

Note: 0x25 clears only that slot's pipeline storage area; the color data area is untouched. To clear color + pipeline together, use 0x14 (one slot) or 0x15 (all).

8. Slot Playback Parameter Configuration (0x1B)

Set the playback parameters of the specified slot (playback mode, direction, delays, etc.).

Data payload (8 bytes):

Offset	Length	Description
0	1 B	Slot number (0–19)
1	1 B	Playback mode (see 6.4 Playback Mode)
2	1 B	Playback direction (see 6.5 Playback Direction)
3 ~ 4	2 B	Group start delay (little-endian, ×1000 ms = seconds, range 0–1000)
5 ~ 6	2 B	Group switch delay (little-endian, ×1000 ms = seconds, range 0–1000)
7	1 B	Pipeline delay in frames (range 0–15, default 1)

Note: When the slot configuration is sent to the broadcast address, devices are distinguished by the GroupNo field. Each device saves only the configuration for its own group number.

Delay scope: When multiple devices play in sync, the group start delay and group switch delay only affect the master device's (address 0x01) own playback rhythm; the other devices refresh immediately upon receiving the play command.

9. Responses and Error Handling

9.1 Response Frame Format

After receiving a command, the device replies with a response frame. The response frame has the same structure as the request frame, but DestAddr and SrcAddr are swapped:

Field	Value
DestAddr	The request frame's SrcAddr (sent back to the requester)
SrcAddr	Local device address
Cmd	ACK command (0x80 ~ 0x85)

Field	Value
DataLen	0 or the error-code length

9.2 Error Codes

When `ACK_FAIL (0x81)`, the data payload contains a 1-byte error code:

Error code	Name	Description
0x00	No error	
0x01	Segment number out of range	
0x02	Chip address error	
0x03	Group number error	
0x04	Data length error	
0x05	Invalid color value	
0x06	Hardware error	Storage read/write failure, etc.
0x07	Chip count error	
0x08	Group count error	
0x0A	Parameter error (generic)	

9.3 Timeout and Retry

- The sender should receive a response within **500 ms** of sending a command
- If no response arrives, retry (up to 3 retries recommended)
- Broadcast commands (`DestAddr = 0xFF`) **do not** get responses; no need to wait

9.4 Error Handling Flow

Send command

└ ACK_SUCCESS (0x80) received → command succeeded, continue to the next step

└ ACK_FAIL (0x81) received → check the error code; abort, or correct and resend

└ ACK_BUSY (0x82) received → device busy; wait and retry

└ ACK_CRC_ERROR (0x84) received → recalculate the CRC and resend

└ Timeout → retry (up to 3 times); report an error if still failing

10. Typical Operation Flows

10.1 Send Color Data and Play

1. [Optional] PC → device: stop playback (0x17)
2. PC → device: set slot configuration (0x1B)
← device: ACK_SUCCESS
3. PC → device: send color data - intermediate frame (0x13, chipAddr=0, flags=0x00)
← device: ACK_SUCCESS
4. PC → device: send color data - last frame (0x13, chipAddr=1, flags=0x01)
← device: ACK_SUCCESS (data written to storage)
5. PC → device: start playback (0x16, broadcast)
← device: ACK_SUCCESS (playback started)

10.2 Delete a Single Slot's Data

1. PC → device: delete slot (0x14, slot=3)
← device: ACK_SUCCESS

10.3 Clear All Data

1. [Optional] PC → device: stop playback (0x17)
2. PC → device: clear all colors (0x15)
← device: ACK_SUCCESS

11. Firmware Upgrade (OTA)

Devices support online firmware upgrade via **USB CDC (virtual serial port)** or **RS485** without a programmer. The four commands defined in this chapter (0x26 ~ 0x29) are **standalone commands**; developers can call them individually from any host program that supports this protocol (see [2. Frame Format](#)) to download the BIN image file we provide to the device and complete the upgrade.

11.1 BIN Image Specifications and CRC Calculation

Before sending, the sender calculates the following two parameters from the BIN file:

Parameter	Source	Description
image_size	BIN file size in bytes	Just read the file length; must be ≤ 98304 (96 KB)
image_crc	Modbus CRC16 over the entire BIN byte stream	Initial value 0xFFFF, polynomial 0xA001, 2-byte result

image_crc reference implementation (identical to 2.3 CRC16 Verification):

```
/* C: computed over the entire BIN file byte stream */
uint16_t OTA_ImageCRC16(const uint8_t *bin, uint32_t len) {
    uint16_t crc = 0xFFFF;
    for (uint32_t i = 0; i < len; i++) {
        crc ^= bin[i];
        for (uint8_t j = 0; j < 8; j++) {
            crc = (crc & 1) ? ((crc >> 1) ^ 0xA001) : (crc >> 1);
        }
    }
    return crc; /* send with low byte first */
}
```

Note: `image_crc` is the CRC over the **entire BIN file** — a different thing from each protocol frame's own frame CRC (section 2.3); do not confuse them.

11.2 OTA Command Details

- Control fields:** For OTA commands (0x26–0x29) the device validates only the data length and does **not** validate `DeviceType` / `GroupNo` / `GroupCount` / `ChipCount` / `ChipAddr` / `SegStart`; these fields may be filled with `00` or reuse the example values.
- Destination address:** Always fill in the **actual address of the target device** for a **point-to-point, single-device** upgrade; **broadcast 0xFF is forbidden** (multiple devices on the bus would respond simultaneously, responses would collide, and they would be indistinguishable).

11.2.1 Start Upgrade (0x26)

Start an upgrade reception, passing the image size and expected CRC. The device erases the staging area and enters the receiving state.

Data payload (6 bytes):

Offset	Length	Description
0 ~ 3	4 B	image_size (little-endian, BIN file size in bytes, ≤ 98304)
4 ~ 5	2 B	image_crc (little-endian, Modbus CRC16 of the BIN file)

Send example (image size = 0x0000D300 = 53984 bytes, CRC = 0x1234):

Header	Dst	Src	Type	Cmd	Len	GN	GC	CC	CA	SS	Data	CRC
Tail												
AA 55	01	00	00	26	06	01	01	02	00	00	00 D3 00 00 34 12	XX XX
55 AA												
└─size(LE32)=53984─┐ └─crc(LE16)=0x1234─┘												

Device response:

Response	Description
ACK_SUCCESS (0x80)	Staging area erased, receiving state entered; OK to start sending 0x27
ACK_FAIL (0x81) + 0x0A	Parameter error: image_size is 0 or > 96 KB; or staging-area erase failed
ACK_FAIL (0x81) + 0x04	Data length error: DataLen < 6

11.2.2 Image Data (0x27)

Send the BIN file in frames of data chunks. Each frame carries one chunk and its byte offset within the image.

Data payload (5 + N bytes, N ≤ 60):

Offset	Length	Description
0	1 B	flags (bit0 = LAST_FRAME; the device does not currently depend on this bit — verification is triggered by 0x28; fill reserved bits with 0)
1 ~ 4	4 B	offset (little-endian, this chunk's starting byte offset within the image; 0 for the first frame)
5 ~ 5+N-1	N B	Data chunk; 60 bytes per chunk recommended (upper limit 61; 60 leaves headroom)

Chunking rule: Start from offset = 0 and send sequentially with +60 bytes per frame until the last chunk (which may be fewer than 60 bytes). The offset field is reserved for resume support; in the first version, simply send in order.

Send example (first chunk, offset = 0, 60 bytes of data):

```

Header  Dst Src Type Cmd Len GN GC CC CA SS  Data
CRC    Tail
AA 55   01 00 00   27  41  01 01  02  00 00  00 00 00 00  [60-byte BIN data]
XX XX   55 AA
                                     Llen=65L LflagsL Loffset(LE32)=0L
└─chunk(60B)─┘

```

DataLen = 5 (flags + offset) + 60 (chunk) = 65 (0x41).

Device response:

Response	Description
ACK_SUCCESS (0x80)	Chunk written to the staging area; OK to send the next frame (waiting for each ACK before the next frame is recommended)
ACK_FAIL (0x81) + 0x0A	Parameter error: data sent without a prior 0x26 / offset out of range / storage write failed

Response	Description
ACK_FAIL (0x81) + 0x04	Data length error: DataLen < 5

11.2.3 End Upgrade (0x28)

Notify the device that the image has been fully sent. The device reads back the whole package, computes the CRC, and compares it against the image_crc passed with 0x26.

Data payload: Empty (DataLen = 0)

Send example:

Header	Dst	Src	Type	Cmd	Len	GN	GC	CC	CA	SS	CRC	Tail
AA 55	01	00	00	28	00	01	01	02	00	00	XX XX	55 AA

Device response and behavior:

Case	Response / behavior
CRC verification passed	The device resets immediately and sends no ACK; the bootloader takes over the copy/upgrade. The sender can detect the device briefly dropping off and then returning to know it was triggered
Incomplete (byte count short)	ACK_FAIL (0x81) + 0x06 (hardware error), state returns to IDLE
CRC mismatch	ACK_FAIL (0x81) + 0x06, state returns to IDLE; may restart with 0x26
No prior 0x26	ACK_FAIL (0x81) + 0x06

Note: Receiving ACK_FAIL + 0x06 means the image data is wrong or the transfer was incomplete — **no upgrade happens**; resend the entire package starting from 0x26 (the device erases the staging area again).

11.2.4 Query OTA Status (0x29)

Query the current OTA reception state (usable to check whether it is receiving, or to verify after an upgrade). The first version is read-only state.

Data payload: Empty (DataLen = 0)

Send example:

Header	Dst	Src	Type	Cmd	Len	GN	GC	CC	CA	SS	CRC	Tail
AA 55	01	00	00	29	00	01	01	02	00	00	XX XX	55 AA

Device response: `ACK_SUCCESS` (`0x80`) + 7 bytes of data:

Offset	Length	Description
0	1 B	Status code (see table below)
1 ~ 4	4 B	<code>image_size</code> (little-endian, the value passed with <code>0x26</code>)
5 ~ 6	2 B	<code>image_crc</code> (little-endian, the value passed with <code>0x26</code>)

Status code	Name	Description
<code>0x00</code>	IDLE	Idle (not started, or back to initial after finishing/failing)
<code>0x01</code>	RECEIVING	Receiving image data
<code>0x02</code>	READY	Verified and about to reset (very short-lived, rarely readable)

11.3 Typical Call Flow

1. Read the BIN file; the sender computes `image_size` and `image_crc` (see 11.1)
2. Host → device: start upgrade (`0x26`, size, crc)
 ← wait for `ACK_SUCCESS`; abort on failure (`0x0A` / `0x04`)
3. for offset in range(0, `image_size`, 60): # 60 bytes per chunk
 chunk = bin[offset : offset + 60]
 host → device: image data (`0x27`, flags=0, offset, chunk)
 ← wait for `ACK_SUCCESS`; abort on failure (or retry this frame)
4. Host → device: end upgrade (`0x28`)
 ← on `ACK_FAIL(0x06)`: CRC mismatch / incomplete data; go back to step 2 and resend
 ← if no response: the device has reset, the upgrade was triggered; go to step 5
5. Wait 1-3 s for the device to finish the bootloader copy and start the new firmware
 (USB re-enumerates; on RS485, wait for the device to respond again)
6. Host → device: query status (`0x29`), or send any command to probe
 ← a normal response = the new firmware is running; upgrade succeeded

11.4 Precautions

- **Single-device upgrade:** upgrade one device at a time on the RS485 bus; fill `dest_addr` with that device's address; **broadcast is forbidden**. Upgrade multiple devices one by one.
- **Keep power stable:** keep the device powered during the upgrade and the bootloader copy. If power is lost during the copy, the bootloader re-copies from the staging area on the next power-up based on the upgrade flag (self-healing), but frequent power loss should still be avoided.
- **Full resend on failure:** resume is not currently supported. Any failure (interrupted transfer, CRC mismatch, etc.) requires restarting from `0x26`.

- **No concurrency:** during an upgrade, do not send color data, playback control, or other commands.
- **Do not modify the BIN file:** use the `.bin` file we provide as-is; do not change the extension, do not concatenate, do not use `.hex`.
- **On repeated CRC failures:** first verify the sender's CRC algorithm (Modbus, initial value 0xFFFF, result low byte first) and that the BIN file is complete.

12. Frame Verification Example

The following is the complete frame example of the "start playback, start slot 0, interval 5 s" command:

```
AA 55          ← frame header
FF            ← destination address: broadcast
00           ← source address: PC
00           ← device type: brick wall
16           ← command: start playback
02           ← data length: 2 bytes
01           ← group number: 1
01           ← group count: 1
02           ← chip count: 2
00           ← chip address: 0
00           ← segment start: 0
00 05        ← data: [start slot = 0][interval = 5 s]
FC 7C        ← CRC16 = 0x7CFC (low byte first, computed per
section 2.3)
55 AA          ← frame tail
```

13. Appendix A: Brick Wall Pattern (0x00) Pixel Mapping

13.1 Physical Layout

The brick wall pattern is an **11 × 11** square grid with 121 pixels in total, driven by 2 driver chips, 64 segments each.

Col:	0	1	2	3	4	5	6	7	8	9	10
y=0	0	11	22	33	44	55	66	77	88	99	110
y=1	1	12	23	34	45	56	67	78	89	100	111
...											
y=10	10	21	32	43	54	65	76	87	98	109	120

13.2 Chip 0 / Chip 1 Partition

The 121 pixels are split between the first 62 and the remaining 59:

Chip 0 (ChipAddr = 0) — 64-byte array:

Array index	Content
0	0xFE (start marker)
1 ~ 62	Pixel colors for 1-D indexes 0–61 (column 0 through the first 7 pixels of column 5)
63	0xFF (padding)

Chip 1 (ChipAddr = 1) — 64-byte array:

Array index	Content
0	0xFF (padding)
1 ~ 4	1-D indexes 65–62 reversed (the last 4 pixels of column 5 in reverse order)
5 ~ 18	Pixel colors for 1-D indexes 66–79 (column 6 through the first 3 of column 7)
19 ~ 21	Horizontal separator color (0xFF = no refresh)
22 ~ 61	Pixel colors for 1-D indexes 80–119 (rest of column 7 through the first 10 of column 10)
62	0xFE (end marker)
63	1-D index 120 (the last pixel)

Simplified view: array 0 covers the left half of the image, array 1 the right half. The 0xFE bytes at the two ends of the arrays are start/end markers; the other 0xFF bytes are padding for empty positions.

14. Appendix B: Hexagon Pattern (0x01) Pixel Mapping

14.1 Physical Layout

The 54 triangles are arranged in 6 rows with row distribution (top to bottom): **7, 9, 11, 11, 9, 7**.

Row 0 (7):	[51] [50] [44] [43] [35] [34] [24]
Row 1 (9):	[53] [52] [46] [45] [37] [36] [26] [25] [9]
Row 2 (11):	[55] [54] [48] [47] [39] [38] [28] [27] [11] [10] [8]
Row 3 (11):	[56] [57] [49] [41] [40] [30] [29] [13] [12] [6] [7]
Row 4 (9):	[58] [59] [42] [32] [31] [15] [14] [4] [5]
Row 5 (7):	[60] [61] [33] [17] [16] [2] [3]

The numbers in brackets are the **array indexes** (0–63) at those positions. Rows run left to right; inter-row centering offsets form the hexagonal tessellation.

14.2 Array Format

Single 64-byte array; triangle colors are **placed directly at the corresponding index positions**:

Array index	Content
0	0xFE (start marker)
1	0xFF (no triangle 1, padding)
2 ~ 61	Colors of the numbered triangles (54 valid positions in total; the rest filled with 0xFF)
62	0xFF (no triangle 62, padding)
63	0xFE (end marker)

Note: Positions 1, 18–23, 62, 63, etc. have no triangle and are filled with 0xFF. 54 valid triangles in total.

15. Appendix C: Small Honeycomb Hexagon Pattern (0x04) Pixel Mapping

15.1 Physical Layout

The 60 hexagons are arranged in **6 rows × 10 columns**, numbered 1–62, with numbers 12 and 52 missing.

```

Row 0 (top): [62] [61] [60] [59] [58] [57] [56] [55] [54] [53]
Row 1:      [51] [50] [49] [48] [47] [46] [45] [44] [43] [42]
Row 2:      [41] [40] [39] [38] [37] [36] [35] [34] [33] [32]
Row 3:      [22] [23] [24] [25] [26] [27] [28] [29] [30] [31]
Row 4:      [ 1] [13] [14] [15] [16] [17] [18] [19] [20] [21]
Row 5 (bot): [ 2] [ 3] [ 4] [ 5] [ 6] [ 7] [ 8] [ 9] [10] [11]
             ← all rows run left to right →

```

Missing numbers **12** and **52**: they correspond to array indexes 12/52, which are horizontal separator positions filled with 0xFF (no refresh).

15.2 Array Format

Single 64-byte array, **number = array index**:

Array index	Content
0	0xFF (padding)
1 ~ 11	Colors of numbers 1–11 (number 1 is at the head of row 4; numbers 2–11 are row 5)
12	0xFF (horizontal separator / missing number 12)
13 ~ 51	Numbers 13–51
52	0xFF (horizontal separator / missing number 52)
53 ~ 62	Numbers 53–62
63	0xFE (end marker)

16. Appendix D: Generic Type Pixel Mapping

16.1 Array Format

Single 64-byte array, **number** = **array index**:

Array index	Content
0	0xFE (start marker)
1 ~ 62	62 color data bytes (currently implemented as a single solid color; see 5.6 Generic (0x06) Color Data Format)
63	0xFE (end marker)

17. Appendix E: Color Data Packing Quick Reference

Device type	Array	Valid pixels
Brick wall (0x00)	2 × 64B	121 (11×11)
Hexagon (0x01)	1 × 64B	54
Number board (0x03)	1 × 64B	60
Small honeycomb hexagon (0x04)	1 × 64B	60
Generic (0x06)	1 × 64B	62

General rule: 0xFE = array start/end marker, 0xFF = empty/padding/no refresh, 0x00 ~ 0x0F = 16-color palette indexes (see [5.1 Color Encoding](#)).